Processos de Software

Métricas DORA e mapeamento do fluxo de valor

DIM0510 — Turma 01 · Sprint 2 · aula de 28/10

Prof. Fernando · UFRN · 2026.2

Roteiro da aula

Min	Bloco	Pergunta
10	Por que medir a entrega	O que separa equipes que entregam bem?
25	Métricas DORA	O que medir, e como tirar do repositório?
15	Fluxo de valor	Onde o trabalho espera?
5	Entrega	O que a Sprint 2 cobra

> Na aula de 26/10 vimos o pipeline. Nesta aula, como saber se ele está ajudando, e onde o processo trava.

Por que medir a entrega

O programa de pesquisa DORA (DevOps Research and Assessment) acompanha há mais de uma década milhares de equipes e encontrou um resultado que contraria a intuição:

Intuição	O que os dados mostram
entregar rápido aumenta o risco	equipes que entregam com mais frequência também falham menos e se recuperam mais rápido
estabilidade pede entregas raras	lotes pequenos são mais fáceis de testar, revisar e desfazer

- Velocidade (*throughput*) e estabilidade não competem: andam juntas
- Por isso as métricas vêm em dois grupos, e uma equipe olha os dois ao mesmo tempo

📖 **Ref.** DORA — Software delivery performance metrics

As cinco métricas

Grupo	Métrica	Definição
Vazão	**Tempo de espera da mudança** (*change lead time*)	do commit no controle de versão até a mudança rodar em produção
Vazão	**Frequência de implantação** (*deployment frequency*)	quantas implantações por período, ou o intervalo entre elas
Vazão	**Tempo de recuperação** (*failed deployment recovery time*)	quanto leva para recuperar de uma implantação que falhou
Instabilidade	**Taxa de falha de mudança** (*change fail rate*)	fração das implantações que exigiram intervenção imediata (reverter, corrigir às pressas)
Instabilidade	**Taxa de retrabalho** (*deployment rework rate*)	fração das implantações não planejadas, feitas por causa de um incidente

Das cinco, duas são **taxas** (falha e retrabalho: a fração dos eventos que deram errado) e duas são **intervalos** (tempo de espera e recuperação: a duração entre dois instantes); a frequência pode ser uma contagem por período ou o intervalo entre implantações. Essa diferença muda a coleta: uma taxa conta eventos, um intervalo subtrai datas.

A rubrica pede **ao menos três**, com o método de coleta descrito e reproduzível, e a interpretação.

📖 **Ref.** DORA — Software delivery performance metrics

Antes de medir: o que é uma implantação no projeto de vocês?

Sem essa definição, nenhuma das cinco métricas tem número. Escolham uma e escrevam:

Se o projeto...	Uma implantação é
publica automaticamente a cada push na `main` (Render, Vercel)	cada push na `main`
publica imagens no GHCR pelo CI	cada execução bem-sucedida do job de publicação
gera releases ou tags	cada release ou tag
ainda não está no ar	cada merge na `main` com o CI verde (e digam isso)

> O método de coleta começa por essa frase: "Neste projeto, uma implantação é ___, e a contamos com o comando ___."

Tirando os números do repositório

Os três comandos abaixo leem, no repositório, os eventos crus que viram os números: as execuções do CI, os tempos de cada PR e o resultado de cada execução. O GitHub guarda essas datas e conclusões; o `gh` resume, sem a equipe precisar anotar nada à mão.

```
# Frequência: execuções do CI na main (cada push na main é uma implantação no MUSI)
gh run list --branch main --workflow ci.yml --limit 100 --json createdAt,conclusion

# Tempo de espera da mudança: da abertura ao merge de cada PR
gh pr list --state merged --json number,createdAt,mergedAt \
  --jq '.[] | "#\(.number): \(((.mergedAt|fromdate) - (.createdAt|fromdate)) / 60 | floor) min"'

# Taxa de falha: execuções vermelhas na main sobre o total
gh run list --branch main --workflow ci.yml --limit 100 --json conclusion \
  --jq 'group_by(.conclusion) | map({(.[0].conclusion): length}) | add'
```

Métrica	Fonte	Limitação
Frequência	execuções do CI ou releases	conta tentativas, inclusive as que falharam
Tempo de espera	PRs (abertura → merge)	não vê o tempo entre o commit e a abertura do PR
Taxa de falha	CI vermelho na `main`	só capta falhas que os testes pegam
Recuperação	da execução vermelha à próxima verde	só para falhas que o CI detecta

Um exemplo real: o MUSI

Coletado do repositório em 15/10, com os comandos do slide anterior:

Métrica	Valor	Como foi calculado
Frequência	18 implantações em 6 semanas; 15 delas na primeira	execuções do CI na `main` (26/08 a 04/10), sem a cancelada
Tempo de espera	36 e 48 min	os dois PRs integrados (abertura → merge)
Taxa de falha	4 de 18, cerca de 22%	execuções vermelhas na `main`
Recuperação	11 min, 13 min e cerca de 26 h	da execução vermelha à próxima verde

Interpretação:

- A frequência média esconde o padrão: uma semana intensa de montagem e depois quase nada. Média sem distribuição engana
- Três das quatro falhas foram na primeira semana, quando o CI estava sendo montado
- Os dois PRs não tiveram revisão: o tempo de espera é curto porque a revisão foi pulada
- Uma recuperação de 26 h mostra que a falha ficou de um dia para o outro: ninguém estava olhando

O que move cada métrica

Métrica	Práticas que a melhoram
Tempo de espera da mudança	PRs pequenos; revisão no mesmo dia; CI rápido; menos etapas manuais
Frequência de implantação	implantação automática; integrar na `main` todo dia; lotes pequenos
Taxa de falha de mudança	testes no CI; proteção da `main`; revisão; mudanças pequenas, fáceis de entender
Tempo de recuperação	alguém olhando o CI vermelho; reverter com um comando; logs e monitoramento

- Quase todas passam por **lotes pequenos**: é o Primeiro Caminho, da aula de 26/10
- A métrica não se melhora diretamente: melhora-se a prática, e a métrica mostra se funcionou

Como registrar as métricas

Uma tabela por métrica, no `docs/` do grupo:

Campo	Exemplo
Métrica	frequência de implantação
Definição usada	uma implantação é um push na `main` com o CI verde
Período	01/11 a 28/11
Comando ou fonte	`gh run list --branch main --workflow ci.yml --json createdAt,conclusion`
Valor	14 implantações em 4 semanas (3, 5, 2, 4 por semana)
Interpretação	a semana 3 caiu por causa da prova; a tendência se manteve

- Outra pessoa, com o mesmo comando e o mesmo período, deve chegar ao mesmo número: é o "reprodutível" da rubrica
- Valor com distribuição (por semana), e não só a média

Cuidados com as métricas

- **Medir o sistema, não as pessoas.** As métricas descrevem como o processo entrega, e não quem trabalha mais
- **Não comparar equipes.** Contextos diferentes (produto, tamanho, tecnologia) tornam a comparação enganosa
- **Não transformar em meta.** Quando um número vira meta, a equipe aprende a mexer no número (lei de Goodhart): PRs vazios aumentam a frequência sem entregar nada
- **Olhar a tendência.** O valor de uma semana diz pouco; a mudança ao longo da sprint diz mais
- **Sempre com o método.** Um número sem "como foi obtido" não pode ser conferido

📖 **Ref.** DORA — Software delivery performance metrics

Mapeamento do fluxo de valor

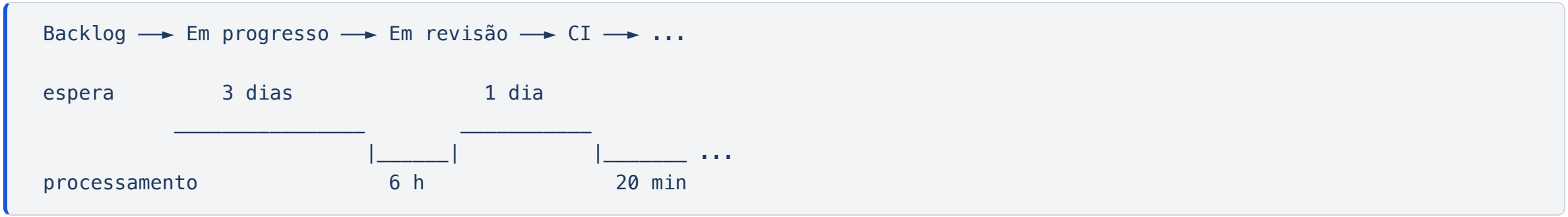
Vem do Lean: desenhar o caminho de um pedido, da ideia até o usuário, e medir quanto tempo ele passa em cada etapa.

Termo	O que é
Tempo de processamento	o tempo em que alguém está de fato trabalhando no item
Tempo de espera	o tempo em que o item está parado, numa fila, esperando alguém
Lead time	do pedido à entrega: a soma de todos os tempos de processamento e de espera
Eficiência de fluxo	tempo de processamento total ÷ lead time × 100
Gargalo	a etapa que limita o fluxo inteiro: onde o trabalho se acumula ou espera mais

> O mapa mostra o que o quadro esconde: um cartão "em revisão" pode estar parado, sem ninguém revisando.

📖 **Ref.** Lean Enterprise Institute — Value-stream mapping

Como desenhar o mapa



Elemento	No desenho
etapa	uma caixa com o nome que a equipe usa
fila entre etapas	a seta entre duas etapas: é onde o item espera
linha do tempo	embaixo, em degraus: em cima as esperas, embaixo os tempos de processamento
totais	no fim da linha: soma das esperas, soma do processamento, lead time

- Papel, quadro branco ou uma ferramenta de desenho: o que importa são as etapas reais e os números

Um mapa, com números

Uma história de usuário de um projeto da disciplina (exemplo ilustrativo):

```
Backlog ──► Em progresso ──► Em revisão ──► CI ──► Merge na main ──► No ar
 espera      trabalho          espera       PT        espera          PT
 3 dias       6 h              1 dia      5 min       2 dias         3 min
            (em 2 dias)       (+ 20 min
                               de revisão)
```

	Horas
Tempo de processamento (6 h + 20 min + 5 min + 3 min)	≈ 6,5 h
Lead time (3 + 2 + 1 + 2 dias)	8 dias = 192 h
Eficiência de fluxo	6,5 ÷ 192 ≈ **3%**

- O maior bloco de espera não é o trabalho: é o item parado antes de começar e depois de aprovado
- O gargalo, com número: depois de começado, o item espera mais entre o CI verde e o merge (2 dias) do que em qualquer outra etapa, porque ninguém é responsável por integrar

Medindo os tempos de verdade

Etapa	De onde vem o tempo
Backlog → em progresso	data de criação do cartão e o campo **Início** do quadro (Sprint 1)
Em progresso	campos **Início** e **Fim** do cartão; o primeiro e o último commit do ramo
Em revisão	abertura do PR → primeira revisão → aprovação
CI	duração de cada execução (`gh run list`)
Merge e implantação	`mergedAt` do PR e o registro da plataforma

O comando abaixo lê, para cada PR já integrado, os três instantes que viram os tempos de espera: a abertura, a primeira revisão e o merge.

```
gh pr list --state merged --json number,createdAt,mergedAt,reviews \
  --jq '.[] | {pr: .number, aberto: .createdAt, primeira_revisao: (.reviews[0].submittedAt // "nenhuma"), merge: .mergedAt}'
```

- O tempo de processamento é o mais difícil: perguntem a quem fez, e anotem que é estimado
- Os tempos de espera são os mais fáceis: as datas já estão no GitHub

Lei de Little, de novo

Da Sprint 1 (Kanban):

> **lead time médio = itens em andamento (WIP) ÷ vazão**

Situação do grupo	Lead time médio
8 cartões em andamento, 2 concluídos por semana	8 ÷ 2 = **4 semanas**
4 cartões em andamento, 2 concluídos por semana	4 ÷ 2 = **2 semanas**

- Com a mesma equipe e a mesma vazão, começar menos coisas encurta a espera de cada uma
- No mapa, o WIP alto aparece como espera: cartões começados e parados
- Limitar o WIP é uma melhoria candidata quando o gargalo é "muita coisa ao mesmo tempo"

Do gargalo à melhoria

A rubrica pede **um gargalo sustentado por número** e **uma melhoria com métrica-alvo**:

	Fraco	Forte
Gargalo	"a revisão demora"	"os PRs esperaram, em média, 26 h pela primeira revisão (7 PRs da Sprint 2)"
Melhoria	"revisar mais rápido"	"revisão no mesmo dia: cada integrante olha os PRs abertos ao fim do dia"
Métrica-alvo	—	"espera pela primeira revisão abaixo de 8 h nas duas semanas seguintes"

- A melhoria é uma mudança no processo, com dono, e não um pedido de esforço
- A métrica-alvo é a mesma que mostrou o gargalo: assim é possível saber se melhorou
- É a mesma estrutura das ações da retrospectiva: responsável, prazo e como verificar

Erros comuns no mapeamento

Erro	Por que é um problema
mapear o processo como deveria ser	o mapa serve para ver o que acontece de fato, inclusive o que ninguém gosta de admitir
anotar só o tempo de trabalho	a espera é quase todo o lead time; sem ela, a eficiência parece ótima
um cartão só	um caso pode ser exceção; com três ou mais, aparece o padrão
esquecer etapas fora do quadro	esperar o cliente, a implantação manual, a aprovação do professor
apontar uma pessoa como gargalo	o gargalo é uma etapa do processo, e a melhoria muda o processo
melhoria sem métrica-alvo	não há como saber, depois, se funcionou

Sugestão para o grupo

Depois da aula, cada grupo faz no próprio quadro e repositório:

1. **Desenhar.** As etapas reais do fluxo, da ideia ao "no ar". As do quadro, e as que o quadro não mostra
2. **Medir.** Escolher três cartões já concluídos e anotar, para cada etapa, o tempo de espera e o de processamento, com a fonte de cada número
3. **Calcular.** Lead time e eficiência de fluxo de cada cartão, e a média
4. **Decidir.** Qual é o gargalo, com o número que o mostra, e qual melhoria testar, com a métrica-alvo

Ao final, a pergunta é: onde o trabalho de vocês mais espera, e quanto?

O que a Sprint 2 cobra destas duas aulas

Critério	Peso	O que entregar
Pipeline de CI e containerização	25%	workflow com build, testes e lint em push e PR; verde na `main`; proteção da `main`; `docker compose up` do zero
Métricas DORA	20%	ao menos 3 métricas, com método reprodutível e interpretação
Mapeamento de Fluxo de Valor	25%	mapa com tempos medidos, eficiência de fluxo, 1 gargalo com número e 1 melhoria com métrica-alvo

- A retrospectiva final (`docs/retrospectiva-02.md`) usa estes números para mostrar a evolução do processo
- Datas: prova em **11/11**; *daily meetings* em 23 e 25/11; **entrega final em 30/11**

> **Prova (11/11):** presencial, no laboratório, no horário da aula, no **Multiprova**, com questões fechadas sobre as Sprints 0 e 1. É permitida **uma folha A4 manuscrita, frente e verso**; nenhum outro material pode ser consultado.

Para conferir o entendimento

1. Por que velocidade e estabilidade andam juntas nos dados do DORA?
2. No projeto de vocês, o que conta como uma implantação?
3. Um cartão levou 10 dias do backlog ao ar, com 8 horas de trabalho. Qual a eficiência de fluxo?
4. Por que transformar a frequência de implantação em meta pode piorar o processo?
5. O grupo tem 9 cartões em andamento e conclui 3 por semana. Qual o lead time médio esperado?

Onde estudar depois

- Leitura da sprint: `leituras/processos-s2.md`, capítulos 5 a 8
- DORA — Software delivery performance metrics
- Lean Enterprise Institute — Value-stream mapping
- GitHub CLI — gh run list · gh pr list